Photon flux

October 12, 2026

The flux is written in `src/photon_flux.cpp` and given as $dN/d\omega$: number of photons per unit photon energy.

1 Variables

The code works with the light-cone momentum of the photon q^+, the photon energy ω and the energy fraction z_γ (the fraction of the beam energy carried by the photon, Petja's tables call it y). They are related by

$$\omega = \frac{q^+}{\sqrt{2}}, \qquad z_\gamma = \frac{2\omega}{\sqrt{s_{NN}}}, \qquad \text{so} \qquad \frac{dN}{d\omega} = \frac{2}{\sqrt{s_{NN}}}\,\frac{dN}{dz_\gamma}, \tag{1}$$

with $\sqrt{s_{NN}} = 5.36$ TeV for Pb+Pb. The flux can also be given as dN/dz_γ, or $z_\gamma\, dN/dz_\gamma$.

2 Different fluxes in this code

The flux is chosen with the environment variable `FLUX_MODEL` (the defaults are in `run_scripts/config.sh`):

- `EFF` (default): "effective" flux of arXiv:2404.09731 (Eq. 4).
- `STARLIGHT`: table made by P. Paakkinen in arXiv:2404.09731 (AnAn, An0n and 0n0n).
- `PL`: flux of a point-like nucleus at a fixed impact parameter b.
- `WS`: flux for a Woods–Saxon charge distribution.

The impact parameter b is the distance between the centres of the two nuclei. The `PL` and `WS` fluxes depend on b, which is integrated by the code. In `EFF` and `STARLIGHT` this integral is already done, so the flux depends only on the photon energy.

Two more settings change the flux:

- `CHANNEL`, the neutron class. The nuclei can break up and emit neutrons, which is measured in the zero degree calorimeters. The probability that a nucleus does not break up is $P_{\text{noEM}}(b) = \exp(-S/b^2)$ with $S = (17.4\text{ fm})^2$. The flux is multiplied by 1 for AnAn (any number of neutrons), by P_{noEM} for An0n, by $P_{\text{noEM}}(1 - P_{\text{noEM}})$ for Xn0n and by P_{noEM}^2 for 0n0n.
- `GAMMA_AA_FILE`, the probability $\Gamma_{AA}(b)$ that the two nuclei do not collide hadronically. It depends on the cross section σ_{NN}. The default file uses $\sigma_{NN} = 92$ mb , `input/WS_photon_flux/Gamma_AA_sigma90.85.dat` uses the 90.85 mb of Petja's tables. One can make a new file with `src/make_gamma_aa.py`.

The Woods–Saxon parameters are $R = 6.49$ fm and $d = 0.54$ fm, the same as in Petja's tables.

In p+Pb (`TARGET=pA`, $\sqrt{s_{NN}} = 8.16$ TeV) the flux is `WS` (default) or `PL`. There are no neutron classes, and $\Gamma_{AA}(b)$ is replaced by $\Gamma_{pA}(b) = \exp[-\sigma_{NN}\, T_A(b)]$ with $\sigma_{NN} = 99$ mb (`SIGMA_NN`).

3 How the flux gets into the calculation

In `src/photon_flux.cpp` there are two functions that give $dN/d\omega$:

- Flux that depends on b (`PL`, `WS`): `photon_flux(b, qp, par)` gives $2\pi b\, \dfrac{d^3N}{d\omega\, d^2b}\, \Gamma_{AA}(b)\, P_{\text{channel}}(b)$.
- Flux already integrated over b (`EFF`, `STARLIGHT`): `effective_photon_flux(qp, par)` gives $\dfrac{dN}{d\omega}$.

The first one still has to be integrated over b; the code does that as one of the variables of its Monte Carlo integration (VEGAS). For a point-like nucleus the flux at fixed b is

$$\frac{d^3N}{d\omega\, d^2b} = \frac{\alpha Z^2}{\pi^2\,\omega}\,\frac{\eta^2}{b^2}\,K_1^2(\eta), \qquad \eta = z_\gamma\, m_N\, b, \tag{2}$$

and the Woods–Saxon and effective fluxes are built from the same quantity. Petja's table contains $\ln(dN/dz_\gamma)$, so the code takes the exponential and multiplies by $2/\sqrt{s_{NN}}$, inside `effective_photon_flux()`.

The cross section is computed in `src/integrands.cpp`, as an integral over q^+. Since $q^+ = \sqrt{2}\,\omega$, it needs the flux per unit q^+,

$$\frac{dN}{dq^+} = \frac{1}{\sqrt{2}}\,\frac{dN}{d\omega}. \tag{3}$$

This is done in `flux_per_qp()`. There the code asks which kind of flux it has (with `is_effective_flux()` in `src/def.hpp`), and then every integrand uses the value in the same way:

```
double dN_domega = is_effective_flux(par) ? effective_photon_flux(qp, p)
                                          : photon_flux(b, qp, p);
return dN_domega / M_SQRT2;                          // in flux_per_qp()
...
double flux = flux_per_qp(b, kin.qp, p);
double kernel = kin.z * m*m * kin.jac_qp * flux * g;   // for example, exclusive
```

Everything else that depends on the photon energy ($z = p^+/q^+$, the photon–nucleus energy W^2, $x_{\mathbb{P}}$, …) is computed here from q^+, and it is the same for every flux. Then, the flux can be changed without modifying the integrands.

Careful: dipole impact parameter b_d

The impact parameter b_d is the position in the target nucleus where the dipole (the charm quark–antiquark pair) interacts. The dipole amplitude depends on b_d, so it is given as Glauber samples, one file per value, `data/Pb/mve/glauber_mve_<`b_d`>` with $b_d = 0, 2, \ldots, 32\ \text{GeV}^{-1}$ (17 files).

The tables (`EFF`, `STARLIGHT`) only remove the integral over the flux b. The integral over b_d is still needed for every flux: `run_scripts/run_nucleus.sh` runs the D^0 program once for each Glauber file, and `plotting_scripts/D0.py` then computes

$$\frac{d\sigma}{dy\,d^2p_{D^0\perp}} = 2\pi \int db_d\, b_d\, \frac{d\sigma(b_d)}{dy\,d^2p_{D^0\perp}} \tag{4}$$

with Simpson's rule (`integrate_over_b_d`). So with a table flux, one D^0 run for a single Glauber file is not the cross section yet but one point of this b_d integral.

4 How to add another flux

4.1 A table in z_γ

The easiest way is to copy what is done for Petja's table (`init_starlight_flux()` and the `STARLIGHT` part of `effective_photon_flux()` in `src/photon_flux.cpp`):

1. Write a function `init_<name>_flux(channel)` that reads your table. In `effective_photon_flux()`, add a part for your flux that returns $dN/d\omega$. If your table has dN/dz_γ, multiply by $2/\sqrt{s_{NN}}$. If it has $z_\gamma\,dN/dz_\gamma$, divide by z_γ first. If it has a logarithm, take the exponential first.
2. Add the name of your flux to `is_effective_flux()` in `src/def.hpp`. Otherwise the code thinks your flux still depends on b and integrates it over b a second time.
3. Call `init_<name>_flux()` where the other fluxes are set up: in the Pb+Pb part of `src/setup.cpp`, and in `src/scan_flux.cpp` to check it.

No need to add factors of z_γ, ω, $1/q^+$ or $1/\sqrt{2}$ for the flux in the integrands.

4.2 A flux that depends on b

Add a part to `photon_flux(b, qp, par)` that returns $2\pi b\,(d^3N/d\omega\,d^2b)$. The factors $\Gamma_{AA}(b)$ and $P_{\text{channel}}(b)$ are applied after it, as for `PL` and `WS`. Do not add this flux to `is_effective_flux()`.

5 Scan flux

The program `build/bin/scan_flux` prints the flux as dN/dz_γ for many values of z_γ, using exactly the same functions as the D^0 calculation:

```
./build/bin/scan_flux STARLIGHT AnAn AnOn     # or EFF, PL, WS, or your flux
```

If the flux is a table, the output must give the table back. To compare several fluxes in one plot, run `run_scripts/run_flux_scan.sh` and then `plotting_scripts/flux_comparison.py` with the same σ_{NN}.